VisionQ

Manual de despliegue en la nube

Contenerización con Docker, ejecución local, integración frontend–backend por API y estrategia de despliegue en contenedores y PaaS

Asignatura: Gestión de proyectos de inteligencia artificial — Máster en Inteligencia Artificial

Actividad: Fase II. Ejecución — solución técnica completa y desplegable

Profesora: Tania Mendoza

Estudiante: Rene

Repositorio: https://github.com/edgdark80-lgtm/VisionQ

Versión: 2.0.0

Fecha: 29 de octubre de 2026

Contenido

1. Introducción y alcance

Este manual describe, de forma replicable, cómo construir, ejecutar, validar y desplegar en la nube la solución **VisionQ**, desarrollada a partir de los modelos construidos en las actividades anteriores del curso. VisionQ atiende dos necesidades de una planta de manufactura automotriz (contexto simulado, sin datos internos): la **inspección visual automática de defectos superficiales** en lámina de acero y la **predicción de fallas de máquina** a partir de lecturas de sensores.

La solución se entrega como una única imagen Docker que contiene el backend (API REST con FastAPI), el frontend web que consume esa API y los dos modelos entrenados. El mismo artefacto se ejecuta sin cambios en una laptop (incluida Apple Silicon M1), en un pipeline de integración continua y en plataformas de contenedores administradas (Google Cloud Run, Render, Azure Container Apps), lo que materializa los principios de **portabilidad** y **consistencia** entre entornos.

Tabla 1. *Componentes de la entrega y su ubicación en el repositorio*

Entregable	Ubicación	Estado
Código fuente completo (backend, frontend, entrenamiento)	`app/`, `frontend/`, `training/`	Completo
Dockerfile funcional y docker-compose	`Dockerfile`, `docker-compose.yml`, `.dockerignore`	Construido y ejecutado
Configuración	`.env.example`, `pyproject.toml`, `requirements*.txt`, `Makefile`	Completo
Evidencia de integración (API, endpoints)	`docs/API.md`, `docs/evidence/`, `/docs` (Swagger)	Generada
Despliegue en la nube	`deploy/`, `.github/workflows/release.yml`	Preparado (requiere cuenta en la nube)
Pruebas automatizadas	`tests/` (46 pruebas), `scripts/`	46/46 aprobadas
Cuaderno para Google Colab	`notebooks/VisionQ_Colab.ipynb`	Ejecutado de inicio a fin

2. Arquitectura de la solución

VisionQ sigue una arquitectura de **servicio único contenerizado** con separación lógica de capas. El frontend es estático y se comunica con el backend exclusivamente mediante la API REST versionada `/api/v1`; por ello puede publicarse junto al backend (opción por defecto, mismo origen, sin problemas de CORS) o por separado en un almacenamiento estático cambiando una sola variable (`frontend/config.js`).

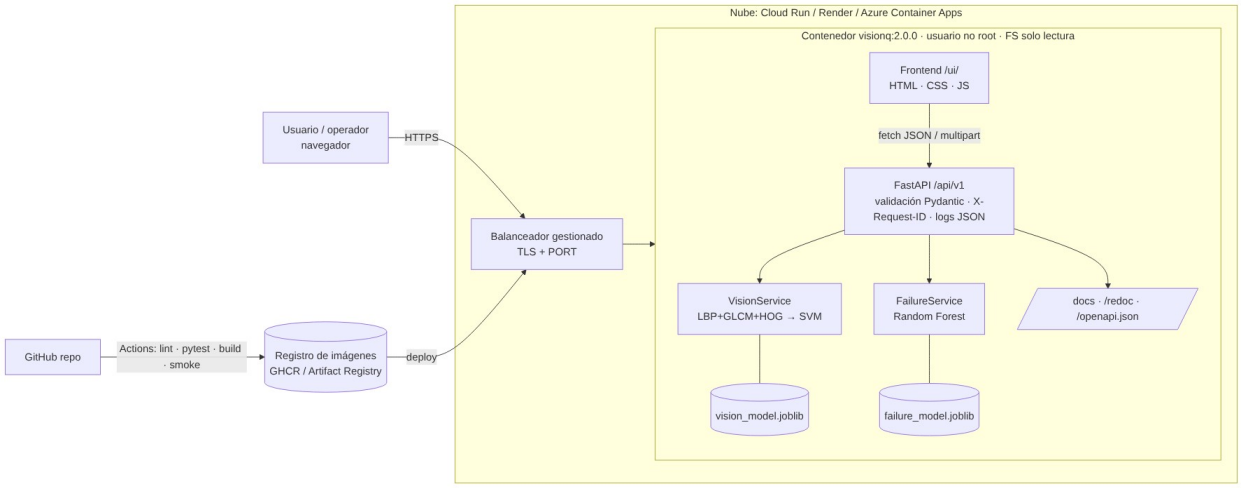


Figura 1. *Arquitectura de despliegue de VisionQ: contenedor único con frontend, API y modelos, alimentado por el pipeline de GitHub Actions.*

Tabla 2. *Responsabilidad de cada componente*

Capa	Tecnología	Responsabilidad
Frontend	HTML5, CSS, JavaScript (sin dependencias)	Carga de imágenes, formulario de sensores, visualización de la disposición, del riesgo y de las métricas del sistema
API	FastAPI 0.162, Uvicorn, Pydantic 2	Validación de entradas, contratos OpenAPI, códigos HTTP, trazabilidad (`X-Request-ID`), logs JSON, métricas, API key opcional
Servicio de visión	scikit-image + scikit-learn (SVM RBF calibrado)	Extracción de 374 descriptores de textura, clasificación en 6 clases, detección fuera de distribución y control de calidad de captura
Servicio de fallas	scikit-learn (Random Forest)	Ingeniería de variables físicas (potencia, ΔT, esfuerzo), probabilidad de falla, indicadores por modo de falla
Modelos	Archivos `.joblib` + métricas `.json`	Artefactos versionados que se copian a la imagen en tiempo de construcción

Una decisión de diseño clave es que la extracción de características vive en un único módulo (`app/features.py`) compartido por el entrenamiento y la inferencia, lo que elimina la desviación entre cómo se entrenó y cómo se sirve el modelo (*training/serving skew*), un riesgo habitual al llevar modelos a producción (Sculley et al., 2015).

3. Requerimientos técnicos

3.1 Ejecución local

Tabla 3. *Requerimientos para construir y ejecutar localmente*

Elemento	Mínimo	Recomendado / notas
Sistema operativo	macOS 12+, Windows 10/11 con WSL2, Linux x86_64/arm64	Probado en Linux x86_64; la imagen base es multi-arquitectura (válida para Apple M1)
Docker	Docker Engine 24+ o Docker Desktop 4.25+	Docker Compose v2 incluido
CPU / RAM	2 núcleos, 4 GB de RAM (1 GB reservado al contenedor)	En iMac M1 de 8 GB funciona con el límite de 1 GB del compose
Disco	≈ 1.5 GB libres	Imagen final: 212 MB comprimida, 885 MB descomprimida
Red	Acceso a Docker Hub y PyPI durante el build	En ejecución no requiere Internet
Opcional (desarrollo)	Python 3.11, Git, make	Para pruebas sin contenedor y reentrenamiento

3.2 Despliegue en la nube

Tabla 4. *Requerimientos para el despliegue en la nube*

Elemento	Detalle
Cuenta	Google Cloud con facturación activa (la capa gratuita de Cloud Run cubre el uso académico), o cuenta en Render / Azure
CLI	`gcloud` (Google Cloud SDK) autenticado; o `az` (Azure CLI) con la extensión `containerapp`
APIs habilitadas	Cloud Run, Artifact Registry, Cloud Build (el script las habilita automáticamente)
Recursos por instancia	1 vCPU, 1 GiB de RAM, concurrencia 8, 0–3 instancias
Puerto	El contenedor escucha en `$PORT` (8000 por defecto); la plataforma lo inyecta
Sonda de salud	`GET /api/v1/health` (HTTP 200)
Secretos (opcional)	`VISIONQ_API_KEY` en Secret Manager o variables cifradas de la plataforma

3.3 Variables de configuración

La aplicación sigue la metodología *twelve-factor*: toda la configuración se inyecta por variables de entorno con prefijo `VISIONQ_`, sin reconstruir la imagen (Wiggins, 2017).

Tabla 5. *Variables de entorno soportadas*

Variable	Valor por defecto	Efecto
`VISIONQ_CONFIDENCE_THRESHOLD`	0.60	Debajo de este valor la inspección se deriva a `REVISION_MANUAL`
`VISIONQ_FAILURE_THRESHOLD`	(vacío → 0.36)	Umbral de decisión de falla; vacío usa el seleccionado en validación
`VISIONQ_MAX_UPLOAD_MB`	5	Tamaño máximo de imagen (HTTP 413 si se excede)
`VISIONQ_MAX_BATCH_IMAGES` /	16 / 1000	Límites de los endpoints por lote

Variable	Valor por defecto	Efecto
`_RECORDS`		
`VISIONQ_API_KEY`	(vacío)	Si se define, los `POST` exigen el encabezado `X-API-Key` (HTTP 401)
`VISIONQ_CORS_ORIGINS`	*	Orígenes permitidos si el frontend se aloja aparte
`VISIONQ_LOG_LEVEL` / `_ENVIRONMENT`	INFO / local	Nivel de log y etiqueta de entorno visible en `/health`
`PORT` / `WEB_CONCURRENCY`	8000 / 1	Puerto y número de procesos Uvicorn

4. Construcción del contenedor

4.1 Dockerfile

El `Dockerfile` usa una construcción **multi-etapa**: la etapa `builder` instala las dependencias fijadas en un entorno virtual aislado y la etapa `runtime` copia únicamente ese entorno, el código, el frontend y los modelos. Así la imagen final no contiene compiladores, cachés de pip, datos de entrenamiento, pruebas ni documentación (controlado además por `.dockerignore`).

```
ARG BASE_IMAGE=python:3.11-slim
FROM ${BASE_IMAGE} AS builder
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
COPY requirements.txt .
RUN pip install --upgrade pip && pip install -r requirements.txt

FROM ${BASE_IMAGE} AS runtime
ENV PYTHONUNBUFFERED=1 PATH="/opt/venv/bin:$PATH" PORT=8000 WEB_CONCURRENCY=1
RUN useradd --create-home --uid 1000 visionq
WORKDIR /app
COPY --from=builder /opt/venv /opt/venv
COPY --chown=visionq:visionq app/ ./app/
COPY --chown=visionq:visionq models/ ./models/
COPY --chown=visionq:visionq frontend/ ./frontend/
USER visionq
EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=5s --start-period=20s --retries=3 \
  CMD python -c "...urlopen('http://127.0.0.1:$PORT/api/v1/health')..."
CMD ["sh","-c","exec uvicorn app.main:app --host 0.0.0.0 --port ${PORT} --workers $
{WEB_CONCURRENCY} --proxy-headers"]
```

Tabla 6. *Buenas prácticas aplicadas en la imagen*

Práctica	Implementación	Beneficio
Imagen base mínima	`python:3.11-slim`, multi-arquitectura	Menor superficie de ataque; corre en amd64 y arm64 (M1)
Multi-etapa	`builder` → `runtime`	Imagen final sin herramientas de compilación
Dependencias fijadas	Versiones exactas en `requirements.txt`	Builds reproducibles
Usuario no root	`USER visionq` (UID 1000)	Principio de mínimo privilegio
Sistema de archivos de solo lectura	`--read-only --tmpfs /tmp`	Inmutabilidad; el contenedor no puede alterarse en ejecución
HEALTHCHECK sin dependencias	Sonda en Python estándar	Docker y las plataformas detectan instancias no sanas
Puerto configurable	`$PORT` inyectado por la plataforma	Compatible con Cloud Run, Render, App Service, Railway
Logs a stdout en JSON	`PYTHONUNBUFFERED=1` + formateador JSON	Ingesta directa en Cloud Logging / Azure Monitor
Metadatos OCI	Etiquetas `org.opencontainers.image.*`	Trazabilidad de versión y repositorio

4.2 Comandos de construcción

```
git clone https://github.com/edgdark80-lgtm/VisionQ.git
cd VisionQ
docker build -t visionq:2.0.0 .          # o: make build
docker images visionq
```

En la validación realizada, la construcción completa sin caché tardó **1 min 25 s** y produjo la imagen `visionq:2.0.0` de **212 MB comprimidos** (885 MB descomprimidos). El registro íntegro se conserva en `docs/evidence/docker_build.log`.

Nota para Apple Silicon: la imagen se construye nativamente en arm64. Si se requiere una imagen amd64 para la nube desde una Mac M1, se usa `docker buildx build --platform linux/amd64 -t visionq:2.0.0 .`, o bien se delega el build a Cloud Build (sección 8), que es la opción recomendada.

5. Ejecución y validación local

5.1 Arranque del contenedor

```
docker run -d --name visionq -p 8000:8000 --read-only --tmpfs /tmp visionq:2.0.0
docker ps                                  # STATUS: Up ... (healthy)
docker logs -f visionq                     # logs estructurados
# Alternativa declarativa con límites de CPU/RAM:
docker compose up --build -d
```

El estado `healthy` aparece unos 10 segundos después del arranque, cuando la sonda interna recibe HTTP 200 de `/api/v1/health`. Evidencia (`docs/evidence/docker_runtime.txt`):

```
CONTAINER ID   IMAGE           STATUS                    PORTS                     NAMES
9fa3b879704b   visionq:2.0.0   Up 5 minutes (healthy)   0.0.0.0:8000->8000/tcp   visionq
```

5.2 Prueba de humo E2E

El script `scripts/smoke_test.sh` ejecuta 11 verificaciones contra el contenedor en ejecución (incluyendo rutas de error) y guarda cada respuesta en `docs/evidence/`. Termina con código distinto de cero si alguna falla, por lo que se reutiliza en CI y después de cada despliegue en la nube.

```
$ bash scripts/smoke_test.sh http://localhost:8000
  [OK]    GET  /api/v1/health -> HTTP 200
  [OK]    GET  /api/v1/models -> HTTP 200
  [OK]    GET  /ui/ (frontend) -> HTTP 200
  [OK]    GET  /openapi.json -> HTTP 200
  [OK]    POST /api/v1/inspect (scratches) -> HTTP 200
  [OK]    POST /api/v1/inspect/batch (3 img) -> HTTP 200
  [OK]    POST /api/v1/failure/predict (normal) -> HTTP 200
  [OK]    POST /api/v1/failure/predict (riesgo) -> HTTP 200
  [OK]    POST /api/v1/failure/predict (inválido -> 422) -> HTTP 422
  [OK]    POST /api/v1/inspect (texto -> 415) -> HTTP 415
  [OK]    GET  /api/v1/metrics -> HTTP 200
== Resultado: 11 OK, 0 fallas
```

5.3 Validación visual del frontend

Con el contenedor activo, el frontend queda disponible en `http://localhost:8000/ui/`. Las capturas siguientes se generaron automáticamente con Playwright (`scripts/capture_ui.py`) contra el contenedor.

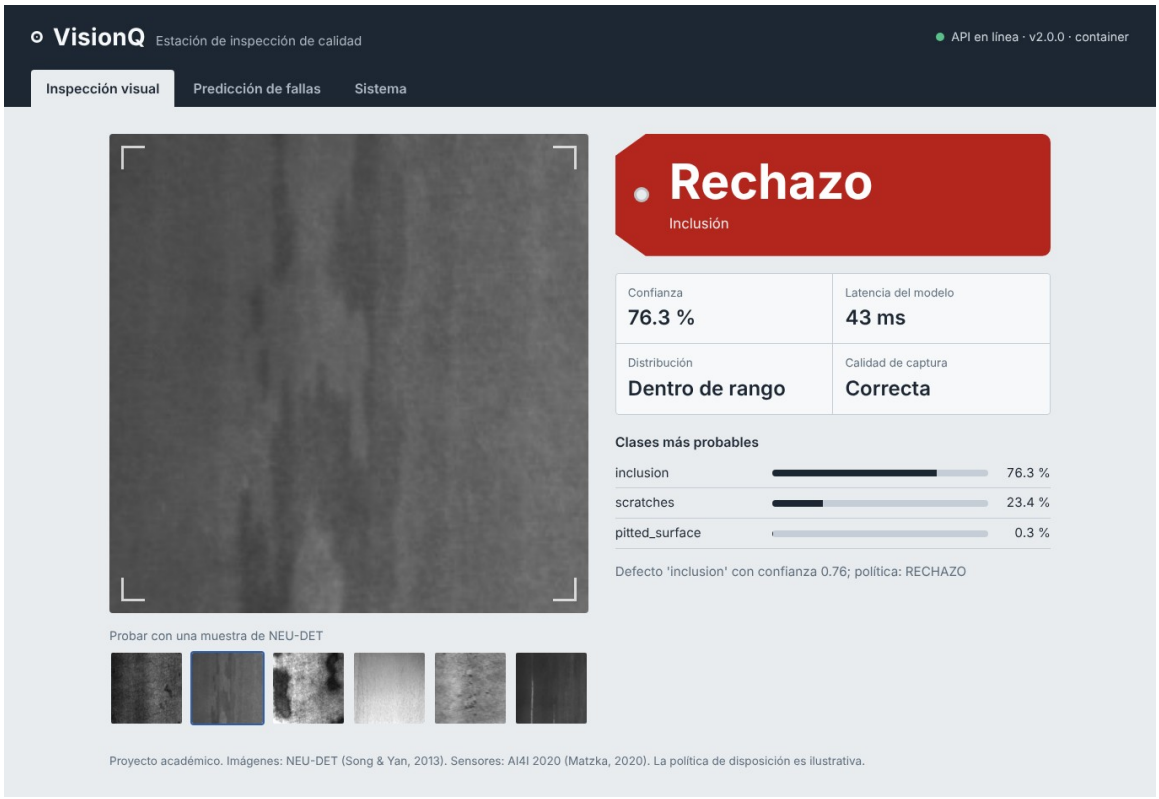


Figura 2. *Inspección visual: el frontend envía la imagen a POST /api/v1/inspect y muestra la disposición, la confianza, la latencia, la verificación de distribución y las clases más probables.*



Figura 3. *Predicción de fallas: lectura de sensores enviada a POST /api/v1/failure/predict con probabilidad, umbral e indicadores por modo de falla.*

6. Integración frontend–backend mediante API

6.1 Contrato de la API

La API está versionada bajo `/api/v1` y su contrato OpenAPI 3.1 se genera automáticamente a partir del código y los esquemas Pydantic, por lo que la documentación nunca se desincroniza de la implementación. Está disponible en `/docs` (Swagger UI, interactiva), `/redoc` y `/openapi.json`.

Tabla 7. *Endpoints expuestos por el backend*

Método	Ruta	Propósito	Códigos
GET	`/api/v1/health`	Estado del servicio y de los modelos	200
GET	`/api/v1/models`	Versión, métricas y umbrales de los modelos	200
GET	`/api/v1/metrics`	Contadores, errores, latencia media, decisiones	200
POST	`/api/v1/inspect`	Clasifica una imagen y emite la disposición	200, 400, 401, 413, 415, 422
POST	`/api/v1/inspect/batch`	Hasta 16 imágenes; errores por archivo no detienen el lote	200, 401, 413, 422
POST	`/api/v1/failure/predict`	Probabilidad de falla de una lectura	200, 401, 422
POST	`/api/v1/failure/predict/batch`	Hasta 1000 lecturas	200, 401, 413, 422
GET	`/ui/`	Frontend que consume los endpoints anteriores	200

Figura 4. *Documentación interactiva generada automáticamente (Swagger UI en /docs).*

6.2 Flujo de comunicación

1. El usuario carga una imagen o un formulario en el frontend (`frontend/app.js`).
2. El frontend envía `multipart/form-data` (imágenes) o JSON (lecturas) con `fetch` a la URL base definida en `frontend/config.js` (vacía = mismo origen).
3. Un *middleware* asigna o propaga `X-Request-ID`, mide el tiempo y escribe un log JSON por solicitud.
4. Pydantic valida tipos, rangos físicos y coherencia (p. ej., temperatura de proceso ≥ temperatura del aire); los errores devuelven 422 con el campo y el mensaje en español.
5. El servicio correspondiente ejecuta el modelo en un *threadpool* (no bloquea el bucle de eventos) y aplica la política de decisión.
6. La respuesta JSON incluye la decisión, la explicación (`decision_reason`, `risk_indicators`) y la latencia; el frontend la representa.

6.3 Ejemplos de consumo

```
curl -F "file=@samples/scratches_195.jpg;type=image/jpeg"
http://localhost:8000/api/v1/inspect
{"filename":"scratches_195.jpg","defect_class":"scratches","defect_name_es":"Rayaduras",
 "confidence":0.843,"decision":"RETRABAJO","out_of_distribution":false,
 "quality":{"width":200,"height":200,"flags":[]},"latency_ms":66.62, ...}

curl -X POST http://localhost:8000/api/v1/failure/predict -H "Content-Type:
application/json" \
  -d '{"type":"L","air_temperature_k":302,"process_temperature_k":310.5,
       "rotational_speed_rpm":1300,"torque_nm":60,"tool_wear_min":210}'
{"failure_probability":0.9486,"threshold":0.36,"prediction":"FALLA_PROBABLE",
 "risk_level":"ALTO","risk_indicators":[{"mode":"HDF",...},{"mode":"OSF",...},
{"mode":"TWF",...}]}
```

7. Estrategia de despliegue en la nube

7.1 Alternativas evaluadas

Tabla 8. *Comparación de estrategias de despliegue*

Criterio	Cloud Run (contenedor serverless)	Render (PaaS)	Azure Container Apps	VM / Kubernetes
Artefacto	Imagen Docker	Dockerfile del repo	Imagen Docker	Imagen Docker
Escalado	Automático, a cero	Manual/automático por plan	Automático, a cero (KEDA)	Manual / HPA
HTTPS y dominio	Incluido	Incluido	Incluido	A configurar
Costo académico	Capa gratuita generosa	Plan gratuito duerme; starter de pago	Capa gratuita mensual	Costo fijo por nodo
Esfuerzo operativo	Muy bajo	Muy bajo	Bajo	Alto
Adecuación a VisionQ	**Alta (elegida)**	Alta (alternativa sin CLI)	Alta (entorno corporativo Microsoft)	Excesiva para el volumen

7.2 Estrategia elegida

Se adopta una estrategia **basada en contenedores sobre una plataforma serverless (Google Cloud Run)**, con **Render como alternativa PaaS** y **Azure Container Apps** para organizaciones con estándar Microsoft. La justificación es la siguiente:

- **Portabilidad:** el mismo `Dockerfile` validado localmente se despliega sin modificaciones en las tres plataformas; no hay dependencia de un *runtime* propietario.
- **Carga intermitente:** la inspección en una estación piloto genera tráfico por ráfagas; escalar a cero evita pagar por instancias ociosas.
- **Inferencia ligera en CPU:** los modelos (SVM y Random Forest) responden en 40–70 ms por solicitud sin GPU, por lo que 1 vCPU / 1 GiB basta (ver documento de validación).
- **Operación simplificada:** TLS, balanceo, revisiones inmutables y *rollback* instantáneo los provee la plataforma.

7.3 Flujo CI/CD

El repositorio incluye dos *workflows* de GitHub Actions. `ci.yml` se ejecuta en cada *push* y *pull request*: lint con Ruff, 46 pruebas pytest, construcción de la imagen, arranque del contenedor y prueba de humo E2E. `release.yml` se activa al crear una etiqueta `v*`: publica la imagen multi-arquitectura (amd64/arm64) en GitHub Container Registry y, si existen los secretos `GCP_PROJECT_ID` y `GCP_SA_KEY`, despliega en Cloud Run y ejecuta la prueba de humo contra la URL pública. Esta automatización es la base de la Fase III (MLOps/GitOps).

8. Procedimiento de despliegue paso a paso

8.1 Google Cloud Run (recomendado)

1. Instalar Google Cloud SDK y autenticarse: `gcloud auth login`.
2. Crear o seleccionar un proyecto con facturación activa y anotar su ID.
3. Desde la raíz del repositorio, ejecutar el script (habilita APIs, crea el repositorio de Artifact Registry, construye la imagen en Cloud Build, despliega y lanza la prueba de humo):

```
PROJECT_ID=mi-proyecto REGION=us-central1 bash deploy/gcp-cloud-run.sh
```

Equivalente manual de los pasos que automatiza el script:

```
gcloud config set project mi-proyecto
gcloud services enable run.googleapis.com artifactregistry.googleapis.com
cloudbuild.googleapis.com
gcloud artifacts repositories create visionq --repository-format=docker --location us-
central1
gcloud builds submit --tag us-central1-docker.pkg.dev/mi-proyecto/visionq/visionq:2.0.0 .
gcloud run deploy visionq-api \
  --image us-central1-docker.pkg.dev/mi-proyecto/visionq/visionq:2.0.0 \
  --region us-central1 --allow-unauthenticated --port 8000 \
  --cpu 1 --memory 1Gi --concurrency 8 --min-instances 0 --max-instances 3 \
  --set-env-vars VISIONQ_ENVIRONMENT=production
URL=$(gcloud run services describe visionq-api --region us-central1 --format
'value(status.url)')
bash scripts/smoke_test.sh "$URL"
```

1. Abrir `$URL/ui/` para el frontend y `$URL/docs` para la API.
2. (Opcional) Proteger la API: crear el secreto `visionq-api-key` en Secret Manager y redeplegar con `--set-secrets VISIONQ_API_KEY=visionq-api-key:latest`.

8.2 Render (PaaS, sin CLI)

1. Iniciar sesión en render.com con la cuenta de GitHub.
2. Seleccionar **New → Blueprint** y elegir el repositorio `edgdark80-lgtm/VisionQ`; Render lee `render.yaml` de la raíz del repositorio (alternativamente, crear un *Web Service* con *Runtime: Docker*).
3. Confirmar el plan (starter recomendado; el plan gratuito suspende el servicio tras inactividad) y aplicar. Render genera automáticamente `VISIONQ_API_KEY`.
4. Esperar a que la sonda `/api/v1/health` reporte el servicio como *Live* y ejecutar `bash scripts/smoke_test.sh https://visionq-api.onrender.com`.

8.3 Azure Container Apps

```
az login
az extension add --name containerapp
RG=rg-visionq LOC=eastus bash deploy/azure-container-apps.sh
```

El script crea el grupo de recursos, un Azure Container Registry, construye la imagen en la nube (`az acr build`), crea el entorno y la aplicación con ingreso externo en el puerto 8000 y escalado 0–3, e imprime la URL pública antes de ejecutar la prueba de humo.

8.4 Publicación de la imagen en GitHub Container Registry

```
git tag v2.0.0 && git push origin v2.0.0
# La imagen queda en ghcr.io/edgdark80-lgtm/visionq:v2.0.0 (amd64 y arm64)
docker run -p 8000:8000 ghcr.io/edgdark80-lgtm/visionq:v2.0.0
```

Alcance de esta entrega: la construcción, ejecución y validación del contenedor se realizaron y documentaron íntegramente en entorno local y en Colab. Los scripts y archivos de despliegue en la nube quedan preparados y parametrizados; su ejecución requiere las credenciales de la cuenta en la nube del estudiante, por lo que no se reportan URLs públicas en este documento.

9. Verificación posterior y operación

Tabla 9. *Lista de verificación posterior al despliegue*

Verificación	Comando / acción	Criterio de aceptación
Salud	`curl $URL/api/v1/health`	`status: ok`, ambos modelos `true`
Prueba de humo	`bash scripts/smoke_test.sh $URL`	11 OK, 0 fallas
Frontend	Abrir `$URL/ui/`	Indicador «API en línea» en verde
Rendimiento	`python scripts/load_test.py --base $URL`	Error 0 %, p95 < 500 ms con concurrencia 8
Logs	`gcloud run services logs read visionq-api`	Logs JSON con `request_id`, ruta, estado y latencia
Métricas de negocio	`curl $URL/api/v1/metrics`	Contadores de decisiones y errores coherentes

9.1 Actualización y reversión

Cada despliegue en Cloud Run crea una revisión inmutable. Para revertir a la revisión anterior sin reconstruir:

```
gcloud run revisions list --service visionq-api --region us-central1
gcloud run services update-traffic visionq-api --region us-central1 --to-revisions
REVISION_ANTERIOR=100
```

Para despliegues graduales (canario) puede asignarse, por ejemplo, el 10 % del tráfico a la nueva revisión con `--to-revisions NUEVA=10` y aumentarlo tras revisar métricas.

10. Seguridad y portabilidad

- **Mínimo privilegio:** proceso como usuario sin privilegios y sistema de archivos de solo lectura.
- **Validación estricta de entradas:** tipo MIME, tamaño máximo, imagen decodificable, rangos físicos y límites de lote; las respuestas de error no exponen trazas internas.
- **Autenticación opcional por API key** (`X-API-Key`) y CORS configurable.
- **Sin secretos en el repositorio ni en la imagen:** `.env` está en `.gitignore` y `.dockerignore`; los secretos se inyectan desde la plataforma.
- **Reproducibilidad:** dependencias con versión exacta, modelos versionados con métricas y fecha de entrenamiento expuestas en `/api/v1/models`.
- **Portabilidad:** imagen multi-arquitectura, puerto por variable, sin dependencias del proveedor en el código.
- **Supervisión humana:** las imágenes de baja confianza, fuera de distribución o con captura deficiente nunca se disponen automáticamente (`REVISION_MANUAL`).

11. Ejecución en Google Colab

Para revisión académica sin instalar nada, el cuaderno `notebooks/VisionQ_Colab.ipynb` (accesible con el botón *Open in Colab* del README) reproduce la solución completa. Colab no permite ejecutar el *daemon* de Docker, por lo que el cuaderno ejecuta el mismo código y los mismos modelos que empaqueta la imagen, servidos con Uvicorn.

1. Abrir `https://colab.research.google.com/github/edgdark80-lgtm/VisionQ/blob/main/notebooks/VisionQ_Colab.ipynb`.
2. Seleccionar **Entorno de ejecución → Ejecutar todas** (runtime de CPU estándar).
3. El cuaderno clona el repositorio, instala las dependencias fijadas en un entorno virtual aislado (sin alterar las librerías de Colab), ejecuta las 46 pruebas, levanta la API, la consume desde Python, evalúa casos extremos, ejecuta la prueba de humo y de carga, abre el frontend mediante el proxy de Colab (`serve_kernel_port_as_window`) y, opcionalmente, reproduce la evaluación de robustez con NEU-DET completo.
4. Tiempo total aproximado: 7 minutos (incluida la evaluación completa).

12. Herramientas de documentación

La documentación técnica se apoyó en herramientas que reducen el esfuerzo manual y mantienen la documentación sincronizada con el código:

Tabla 10. *Herramientas de documentación utilizadas*

Herramienta	Uso en VisionQ
OpenAPI / Swagger UI / ReDoc (FastAPI)	Contrato de la API generado automáticamente desde el código y los esquemas Pydantic; documentación interactiva en `/docs`
Docstrings y comentarios en español	Cada módulo explica su propósito y decisiones de diseño (p. ej., `app/features.py`)
Markdown en el repositorio	`README.md` (inicio rápido) y `docs/API.md` (referencia de endpoints)
Mermaid	Diagrama de arquitectura versionado como código (`docs/arquitectura.mmd`)
Playwright	Capturas de pantalla reproducibles del frontend (`scripts/capture_ui.py`)
Asistentes de IA generativa	Se utilizó un asistente de IA (Claude, de Anthropic) para redactar borradores del README, de este manual y de los comentarios del código, que fueron revisados y validados contra la ejecución real. GitHub Copilot puede emplearse de forma equivalente en el IDE para generar docstrings («/doc») y explicar scripts («/explain») al mantener el proyecto

Como buena práctica, todo contenido generado con asistencia de IA se verificó ejecutando los comandos documentados; las cifras reportadas provienen de los registros de `docs/evidence/` y no de estimaciones.

13. Solución de problemas

Tabla 11. *Problemas frecuentes y solución*

Síntoma	Causa probable	Solución
`Cannot connect to the Docker daemon`	Docker Desktop no está iniciado	Iniciar Docker Desktop y repetir
`port is already allocated`	El puerto 8000 está en uso	`docker run -p 8080:8000 ...` y abrir `localhost:8080`
Contenedor `unhealthy`	Modelos ausentes o RAM insuficiente	`docker logs visionq`; verificar `models/*.joblib` y asignar ≥ 1 GB
`exec format error` en la nube	Imagen arm64 (M1) en nodo amd64	Usar Cloud Build o `docker buildx build --platform linux/amd64`
HTTP 415 al inspeccionar	Archivo con MIME distinto de imagen	Enviar JPEG, PNG, BMP o TIFF con su `Content-Type`
HTTP 401	`VISIONQ_API_KEY` definida	Enviar el encabezado `X-API-Key`
Frontend en «API desconectada»	Frontend alojado en otro origen	Ajustar `frontend/config.js` y `VISIONQ_CORS_ORIGINS`
Primera petición lenta en la nube	Arranque en frío al escalar desde cero	`--min-instances 1` si se requiere latencia constante

Referencias

Docker Inc. (2024). *Dockerfile best practices*. Docker Documentation. https://docs.docker.com/build/building/best-practices/

FastAPI. (2024). *Deployment: FastAPI in containers – Docker*. https://fastapi.tiangolo.com/deployment/docker/

Google Cloud. (2024). *Cloud Run documentation: Deploying container images*. https://cloud.google.com/run/docs/deploying

Matzka, S. (2020). *AI4I 2020 predictive maintenance dataset* [Data set]. UCI Machine Learning Repository. https://doi.org/10.24432/C5HS5C

Merkel, D. (2014). Docker: Lightweight Linux containers for consistent development and deployment. *Linux Journal*, *2014*(239), Artículo 2.

Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., & Dennison, D. (2015). Hidden technical debt in machine learning systems. En *Advances in Neural Information Processing Systems 28* (pp. 2503–2511).

Song, K., & Yan, Y. (2013). A noise robust method based on completed local binary patterns for hot-rolled steel strip surface defects. *Applied Surface Science*, *285*, 858–864. https://doi.org/10.1016/j.apsusc.2013.09.002

Wiggins, A. (2017). *The twelve-factor app*. https://12factor.net/